CIS 5190: Homework 2

Written Answers

Use of AI Tools

I used AI for concepts learning as well as answer checks. I also used it to transform my written answer to Latex format

Time Spent

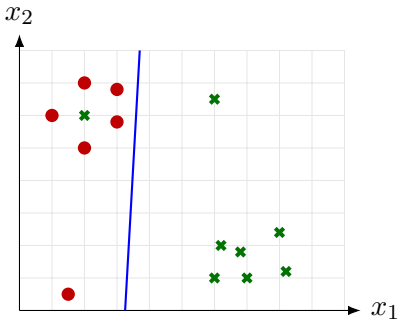
These are rough estimates, not measured times.

Part	Estimated time (hours)
Reading the assignment	0.25
Question 1	0.75
Question 2	0.50
Question 3	0.50
Question 4	0.50
Question 5	0.75
Question 6	0.75
Total	4.00

Question 1: Logistic Regression

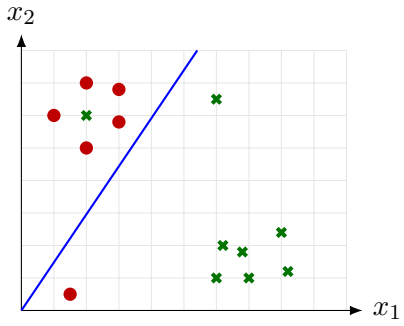
The boundary is $\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0$. Red dots have $y = 1$ and green crosses have $y = 0$. The plots below copy the point positions approximately. The lines show approximate fitted boundaries; the axes use an arbitrary scale.

(a) No strong penalty



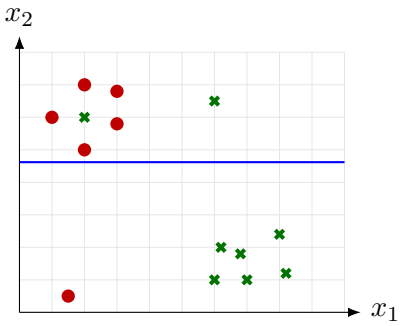
Predict 1 to the left of the line.
1 wrong point: the green cross inside the red group.

(b) Strong penalty on β_0



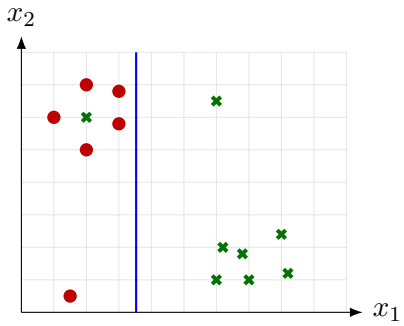
Here $\beta_0 = 0$, so the line must pass through the origin. Predict 1 above the line.
2 wrong points: the green cross inside the red group and the low red dot.

(c) Strong penalty on β_1



Here $\beta_1 = 0$, so the boundary is horizontal. Predict 1 above the line.
3 wrong points: the two high green crosses and the low red dot.

(d) Strong penalty on β_2



Here $\beta_2 = 0$, so the boundary is vertical. Predict 1 to the left of the line.
1 wrong point: the green cross inside the red group.

Question 2: Neural Networks

(a) Zero initialization

All units in the same hidden layer start the same and get the same updates. They cannot learn different features. This is the symmetry problem. With ReLU, zero inputs can also give zero gradients, so hidden units may not learn at all.

(b) Nonlinear activation

Without a nonlinear activation, a stack of linear layers is still one linear map:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

A nonlinear activation lets the network learn nonlinear patterns and boundaries.

(c) Number of parameters

The image has $40 \times 30 = 1200$ input values. Each unit has one bias.

$$\begin{aligned} \text{Hidden layer 1: } & 1200 \times 64 + 64 = 76{,}864, \\ \text{Hidden layer 2: } & 64 \times 32 + 32 = 2{,}080, \\ \text{Output layer: } & 32 \times 10 + 10 = 330. \end{aligned}$$

The total is

$$\boxed{76{,}864 + 2{,}080 + 330 = 79{,}274}.$$

Question 3: k Nearest Neighbors

(a)(i) Two training points

The boundary is the perpendicular bisector of the line segment joining the two points. Every point on this boundary is equally far from the two training points. Each side gets the label of its closer training point.

(a)(ii) Approximating a boundary

Yes, for a usual boundary in a bounded region. Place many training points on both sides of the desired boundary, with the correct label on each side. Put points close to the boundary and close to each other. A finer set of points gives smaller nearest-neighbor regions, so the boundary can follow finer details.

(b) Setting $k = n$

For regression, every query gets the mean of all training targets:

$$\hat{y}(x) = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

For classification, every query gets the most common training label. A fixed rule handles ties. Neither answer depends on the query location.

(c) Bias and variance

A small k gives a flexible boundary. It usually has low bias and high variance because a few training points can change the answer.

A larger k averages over more points. It usually raises bias and lowers variance. At $k = n$, the answer is constant, so local patterns are lost.

Question 4: k Nearest Neighbors for Houses

(a) Different feature scales

Square footage may be in the thousands, while distance to a school may be only a few miles. Raw Euclidean distance is then mostly based on square footage.

Scale each feature, for example with

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

Fit μ_j and s_j on the training data, and use the same values for new data. Feature weights can also help if one feature matters more.

(b) Roof type

Integer IDs create a false order and false distances. For example, IDs 0 and 2 are not necessarily less similar than IDs 0 and 1.

Use one-hot encoding. Each roof type gets its own binary column. Different roof types then have the same distance in the roof feature group. Scale or weight this group as needed.

(c) Curse of dimensionality

In many dimensions, data become sparse. A small local region needs many more samples, and nearest neighbors may no longer be very close. Irrelevant features can also hide useful distances.

Digit images can still work well because their pixels are related. Real digit images occupy a much smaller set than all possible pixel arrays. Their useful dimension can be much lower than the number of pixels.

Question 5: Decision Trees

(a) Why use early stopping?

Growing a large tree takes time and memory. Early stopping avoids many weak splits before they are made. Post-pruning can then remove more branches. A validation set is finite, so pruning may not remove every branch that fits noise.

(b) Bias and variance

These are the usual trends; they are not guaranteed for every dataset.

Change	Usual effect
(i) Increase maximum depth	Lower bias, higher variance. The tree can fit more detail.
(ii) Increase minimum samples to split	Higher bias, lower variance. Small nodes cannot split.
(iii) Disable post-pruning	Lower bias, higher variance. More fitted branches remain.
(iv) Add features to the feature map	Useful features can lower bias. More possible splits can raise variance, especially with noisy features. A tree can ignore new features, so the effect is not certain.

(c)(i) Tree for $A \wedge (B \vee C)$

Each if statement below is a split. Each return value is a leaf.

```
if A == False:
    return False
else:
    if B == True:
        return True
    else:
        if C == True:
            return True
        else:
            return False
```

(c)(ii) Tree for $(A \wedge B) \vee (C \wedge D)$

```
if A == True:
    if B == True:
        return True
if C == True:
    if D == True:
        return True
return False
```

Question 6: Decision Tree Calculations

(a) Class entropy

There are three T examples and one F example. Thus,

$$H(D) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} = \boxed{0.811278}.$$

(b) Information gain

For a_1, the + child has two T examples and entropy 0. The − child has one T and one F, so its entropy is 1.

$$IG(D, a_1) = 0.811278 - \left(\frac{2}{4}\cdot 0 + \frac{2}{4}\cdot 1\right) = \boxed{0.311278}.$$

For a_2, the + child has two T and one F. Its entropy is

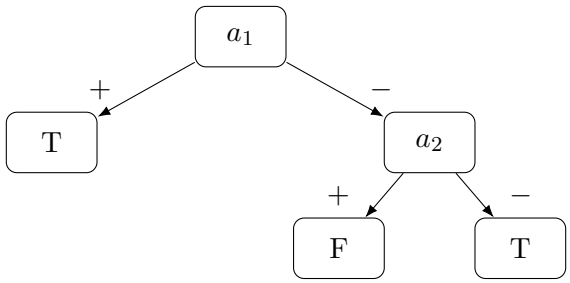
$$H(D_{a_2=+}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.918296.$$

The − child has one T and entropy 0. Thus,

$$IG(D, a_2) = 0.811278 - \left(\frac{3}{4}\cdot 0.918296 + \frac{1}{4}\cdot 0\right) = \boxed{0.122556}.$$

(c) Complete tree

Choose a_1 at the root because it has higher information gain.



(d) New instance

For $a_1 = +$ and $a_2 = -$, the first split goes directly to T. The prediction is $\boxed{\text{T}}$.

(e) Gini gain [CIS 5190]

The parent Gini index is

$$G(D) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375.$$

For a_1, the child Gini indices are 0 and $1 - (1/2)^2 - (1/2)^2 = 0.5$.

$$G_{\text{children},a_1} = \frac{2}{4}(0) + \frac{2}{4}(0.5) = 0.25, \qquad \Delta G(a_1) = 0.375 - 0.25 = \boxed{0.125}.$$

For a_2, the child Gini indices are $1 - (2/3)^2 - (1/3)^2 = 4/9$ and 0.

$$G_{\text{children},a_2} = \frac{3}{4}\cdot\frac{4}{9} + \frac{1}{4}(0) = \frac{1}{3}, \qquad \Delta G(a_2) = \frac{3}{8} - \frac{1}{3} = \frac{1}{24} = \boxed{0.041667}.$$